ECOTRACK

CAHIER DES CHARGES FONCTIONNEL
PLATEFORME INTELLIGENTE DE GESTION DES DÉCHETS URBAINS

📦 Projet Fil Rouge Master
🏢 École INGETIS
📅 Année 2025-2026
⏱️ Durée : 4 mois (Janvier - Avril)
👥 Spécialités : Cyber-Réseaux | Développement | Data Science

Version 3.0 - Janvier 2026

TABLE DES MATIÈRES

1. **I. PRÉSENTATION GÉNÉRALE**

- 1.1 Contexte et enjeux
- 1.2 Vision du projet
- 1.3 Objectifs stratégiques
- 1.4 Bénéfices attendus

2. **II. ANALYSE DU BESOIN**

- 2.1 Problématiques actuelles
- 2.2 Solution proposée
- 2.3 Périmètre fonctionnel
- 2.4 Périmètre technique

3. **III. ACTEURS ET UTILISATEURS**

- 3.1 Parties prenantes
- 3.2 Profils utilisateurs
- 3.3 Cas d'usage principaux

4. **IV. SPÉCIFICATIONS FONCTIONNELLES DÉTAILLÉES**

- 4.1 Module Gestion des Conteneurs
- 4.2 Module Gestion des Zones
- 4.3 Module Optimisation des Tournées
- 4.4 Module IoT et Capteurs
- 4.5 Module Monitoring Temps Réel
- 4.6 Module Analytics et Reporting
- 4.7 Module Gamification Citoyenne
- 4.8 Module Authentification et Droits
- 4.9 Module Support et Communication
- 4.10 Module Administration

5. **V. ARCHITECTURE SYSTÈME**

- 5.1 Architecture globale
- 5.2 Architecture applicative
- 5.3 Architecture données
- 5.4 Architecture réseau et sécurité

6. **VI. MODÈLE DE DONNÉES**

- 6.1 Entités principales
- 6.2 Relations
- 6.3 Contraintes métier

7. **VII. EXIGENCES NON FONCTIONNELLES**

- 7.1 Performance
- 7.2 Sécurité
- 7.3 Disponibilité
- 7.4 Scalabilité
- 7.5 Ergonomie et Accessibilité

8. **VIII. INTERFACES ET INTÉGRATIONS**

- 8.1 Interfaces utilisateurs
- 8.2 API REST
- 8.3 Intégrations externes

9. **IX. ORGANISATION ET MÉTHODOLOGIE**

- 9.1 Méthodologie Agile
- 9.2 Organisation équipes
- 9.3 Planning et jalons

10. **X. LIVRABLES ET DOCUMENTATION**

- 10.1 Livrables techniques
- 10.2 Documentation
- 10.3 Critères d'acceptation

11. **XI. ANNEXES**

- 11.1 Glossaire
- 11.2 Références
- 11.3 Contacts

I. PRÉSENTATION GÉNÉRALE

1.1 Contexte et Enjeux

La gestion des déchets urbains représente un défi majeur pour les métropoles modernes. Face à l'urbanisation croissante et aux enjeux environnementaux, les villes doivent moderniser leur système de collecte pour le rendre plus efficace, écologique et économique.

Constats actuels :

- • Collectes inefficaces avec des passages inutiles sur des conteneurs vides
- • Débordements fréquents de conteneurs dans les zones à forte densité
- • Coûts opérationnels élevés (carburant, main-d'œuvre, maintenance)
- • Impact environnemental important (émissions CO_2, pollution)
- • Manque de visibilité en temps réel sur l'état des conteneurs
- • Engagement citoyen limité dans le tri et la réduction des déchets
- • Planification des tournées peu optimisée
- • Absence de données analytiques pour piloter le service

Enjeux stratégiques :

- 🌍 Environnementaux : Réduire l'empreinte carbone de la collecte
- 💰 Économiques : Optimiser les coûts opérationnels (-20% visé)
- 👥 Sociaux : Impliquer les citoyens dans une démarche éco-responsable
- 📊 Data-driven : Piloter le service par les données
- 🔧 Technologiques : Moderniser l'infrastructure avec l'IoT

1.2 Vision du Projet

ECOTRACK ambitionne de transformer la gestion des déchets urbains en un service intelligent, connecté et participatif, où la technologie IoT, l'intelligence artificielle et l'engagement citoyen convergent pour créer une ville plus propre et durable.

Notre vision repose sur 5 piliers :

Pilier	Description
🔌 **Connectivité IoT**	Conteneurs intelligents avec capteurs temps réel de remplissage
🤖 **Intelligence Artificielle**	Prédiction des remplissages et optimisation automatique des tournées
📱 **Mobilité**	Applications mobiles pour citoyens, agents et gestionnaires
📊 **Analytics**	Dashboards temps réel et rapports analytiques avancés
🎮 **Gamification**	Engagement citoyen par système de points, badges et défis

1.3 Objectifs Stratégiques

Objectif	Indicateur	Cible
Optimiser tournées	Réduction distance parcourue	-20%
Réduire débordements	Taux débordements	<2%
Économiser coûts	Réduction coûts opérationnels	-15%
Réduire émissions	Réduction CO_2	-18%
Engagement citoyen	Utilisateurs actifs	15 000
Disponibilité	Uptime plateforme	>99.5%

1.4 Bénéfices Attendus

Pour la Métropole :

- ✓ Réduction de 20% des coûts de collecte
- ✓ Meilleure qualité de service aux citoyens
- ✓ Données pour pilotage stratégique
- ✓ Image moderne et innovante

Pour les Services de Collecte :

- ✓ Tournées optimisées automatiquement
- ✓ Moins de kilomètres parcourus
- ✓ Application mobile pour agents de terrain
- ✓ Maintenance prédictive des véhicules

Pour les Citoyens :

- ✓ Conteneurs moins souvent pleins
- ✓ Notifications de collecte
- ✓ Engagement ludique via gamification
- ✓ Impact environnemental visible

Pour l'Environnement :

- ✓ Réduction émissions CO_2
- ✓ Optimisation des ressources
- ✓ Sensibilisation au tri
- ✓ Économie circulaire encouragée

II. ANALYSE DU BESOIN

2.1 Problématiques Actuelles

🚛 Collecte Inefficace

Les tournées suivent des circuits fixes sans tenir compte du remplissage réel.

Impacts :

- → Passages inutiles sur conteneurs vides (30-40% des arrêts)
- → Gaspillage carburant et temps
- → Usure prématurée véhicules

⚠️ Débordements Fréquents

Absence de visibilité temps réel sur état des conteneurs.

Impacts :

- → Débordements dans zones touristiques/commerciales
- → Insalubrité et nuisances
- → Plaintes citoyens
- → Interventions d'urgence coûteuses

📊 Manque de Données

Aucune donnée exploitable pour analyser et améliorer le service.

Impacts :

- → Décisions basées sur intuition
- → Impossibilité mesurer performance
- → Pas d'amélioration continue
- → ROI non mesurable

👥 Faible Engagement Citoyen

Les citoyens ne sont pas impliqués dans la gestion des déchets.

Impacts :

- → Tri insuffisant
- → Comportements non responsables

- → Manque de conscience environnementale

2.2 Solution Proposée : ECOTRACK

ECOTRACK est une plateforme IoT intelligente qui transforme la gestion des déchets en un système connecté, optimisé et participatif.

Composantes de la solution :

Composante	**Description**
🔌 **Capteurs IoT**	2000 conteneurs équipés de capteurs ultrasoniques mesurant le taux de remplissage en temps réel. Transmission MQTT vers plateforme cloud.
🗺️ **Optimisation Tournées**	Algorithmes avancés (TSP, 2-opt) pour calculer tournées optimales. Réduction 20% des distances parcourues.
📱 **Applications Mobiles**	Apps pour citoyens (signalements, gamification), agents (tournées guidées) et gestionnaires (monitoring).
📊 **Analytics Avancés**	Dashboards temps réel, prédictions ML, rapports automatisés. KPIs métier et environnementaux.
🎮 **Gamification**	Système points, badges, défis pour engager 15 000 citoyens dans tri et réduction déchets.

2.3 Périmètre Fonctionnel

Modules fonctionnels inclus :

✅ **Gestion Conteneurs**

- CRUD complet, géolocalisation, historique remplissage, alertes

✅ **Gestion Zones**

- Découpage géographique, polygones, statistiques par zone

✅ **Optimisation Tournées**

- Algorithmes TSP/2-opt, création automatique, feuilles de route

✅ **IoT et Capteurs**

- Simulateur MQTT, réception mesures, validation données

✅ **Monitoring Temps Réel**

- Dashboard live, notifications, alertes débordements

✅ **Analytics**

- 8+ types graphiques, évolutions temporelles, exports

✅ **Gamification**

- Points, badges (30+), défis, leaderboard

✅ **Authentification**

- JWT, MFA, RBAC, SSO (optionnel)

✅ **Support**

- Tickets, chatbot, FAQ, signalements citoyens

✅ **Administration**

- Backoffice, gestion users, logs, configuration

Exclusions (hors périmètre Master 1) :

- ❌ Gestion financière et facturation
- ❌ Intégration ERP/SAP de la métropole
- ❌ Application mobile native iOS/Android (PWA suffisant)
- ❌ Blockchain pour traçabilité

- ❌ Computer Vision pour tri automatique
- ❌ Intégration véhicules autonomes

2.4 Périmètre Technique

Technologies et contraintes :

Composant	Technologies
Frontend	React 18+ ou Vue 3 + TypeScript + TailwindCSS
Backend	Node.js + Express ou Python + FastAPI
Base de données	PostgreSQL 14+ + PostGIS pour géospatial
IoT Messaging	Mosquitto MQTT broker
Caching	Redis pour sessions et cache
Maps	Leaflet + OpenStreetMap
Analytics	Chart.js / Recharts / Plotly
CI/CD	GitHub Actions
Déploiement	Docker + Docker Compose
Monitoring	Prometheus + Grafana (Cyber), Logs (Dev/Data)

Contraintes techniques :

- ⚡ Performance : API < 500ms, requêtes SQL < 200ms
- 🔒 Sécurité : HTTPS, JWT, RBAC, protection OWASP Top 10
- 📈 Scalabilité : Support 2000 conteneurs, 15 000 users
- 🔄 Disponibilité : 99.5% uptime visé
- ♿ Accessibilité : WCAG 2.1 niveau AA
- 📱 Responsive : Desktop, tablet, mobile
- 🧪 Tests : >60% couverture code (Dev)
- 📚 Documentation : DCT 30-35 pages par spécialité

III. ACTEURS ET UTILISATEURS

3.1 Parties Prenantes

Partie Prenante	Rôle	Attentes
Métropole	Commanditaire, décideur	ROI, efficacité, conformité réglementaire
Services Techniques	Gestionnaires opérationnels	Outils de pilotage, optimisation tournées
Agents Collecte	Utilisateurs terrain	App intuitive, feuilles route claires
Citoyens	Utilisateurs finaux	Service qualité, engagement valorisé
Étudiants Master 1	Développeurs/Intégrateurs	Projet formateur, technologies modernes

3.2 Profils Utilisateurs

👤 Citoyen

Nombre : 15 000 utilisateurs actifs

Habitant de la métropole engagé dans le tri et réduction déchets.

Besoins fonctionnels :

- • Signaler conteneur plein ou problème
- • Consulter horaires collecte
- • Participer à gamification (points, badges, défis)
- • Voir impact environnemental personnel
- • Accéder conseils tri

Compétences : *Grand public, usage smartphone basique*

🚛 Agent de Collecte

Nombre : 50 agents

Conducteur de camion effectuant tournées quotidiennes.

Besoins fonctionnels :

- • Consulter feuille route optimisée
- • Navigation GPS intégrée
- • Signaler problème terrain (conteneur endommagé, accès bloqué)
- • Marquer conteneur collecté
- • Voir prochains arrêts

Compétences : *Usage basique smartphone, orienté terrain*

👔 Gestionnaire Municipal

Nombre : 20 gestionnaires

Responsable service propreté pilotant opérations.

Besoins fonctionnels :

- • Dashboard temps réel (remplissage, alertes)
- • Créer et modifier tournées
- • Consulter analytics et KPIs

- • Générer rapports (hebdo, mensuel)
- • Gérer zones géographiques
- • Voir historique performances

Compétences : *Bureautique avancée, pilotage par données*

⚙️ Administrateur Système

Nombre : 10 admins

Admin IT gérant plateforme et utilisateurs.

Besoins fonctionnels :

- • Gestion utilisateurs (CRUD, RBAC)
- • Configuration plateforme
- • Logs et audit
- • Monitoring système
- • Gestion conteneurs et zones
- • Support niveau 2

Compétences : *IT expert, administration systèmes*

3.3 Cas d'Usage Principaux

UC-01 - Signaler un conteneur plein

Acteur	Citoyen
Préconditions	Utilisateur connecté à l'application
Scénario	1. Citoyen constate conteneur débordant 2. Ouvre app ECOTRACK sur smartphone 3. Clique "Signaler un problème" 4. Sélectionne conteneur sur carte ou scanne QR code 5. Sélectionne type problème ("Plein", "Endommagé", etc.) 6. Ajoute photo (optionnel) 7. Valide signalement 8. Système crée ticket, notifie gestionnaires 9. Citoyen gagne 10 points
Résultat	Signalement enregistré, équipe notifiée, citoyen récompensé

UC-02 - Optimiser une tournée de collecte

Acteur	Gestionnaire
Préconditions	Conteneurs avec données remplissage à jour
Scénario	1. Gestionnaire accède module "Tournées" 2. Clique "Créer nouvelle tournée" 3. Sélectionne zone géographique 4. Définit critères (remplissage > 70%, type déchet) 5. Système filtre conteneurs éligibles (ex: 45 conteneurs) 6. Gestionnaire choisit algorithme (2-opt recommandé) 7. Système calcule tournée optimale 8. Affiche résultat : distance 32.4 km, durée 4h15, économie 20% 9. Gestionnaire visualise sur carte 10. Valide et assigne tournée à agent 11. Agent reçoit notification avec feuille route
Résultat	Tournée optimisée créée, agent notifié, économie carburant

UC-03 - Effectuer une tournée de collecte

Acteur	Agent de collecte
Préconditions	Tournée assignée, app mobile installée

Scénario	1. Agent ouvre app le matin 2. Consulte tournée du jour (12 arrêts) 3. Lance navigation GPS vers premier conteneur 4. Arrive sur place, scanne QR code conteneur 5. Marque "Collecté" dans app 6. Système met à jour BDD, envoie notification temps réel 7. Agent passe au suivant 8. Si problème (conteneur inaccessible), signale dans app 9. Système recalcule tournée si nécessaire 10. Fin de tournée : agent clôture dans app
Résultat	Tournée complétée, données historiques enregistrées

UC-04 - Analyser performances du service

Acteur	Gestionnaire
Préconditions	Historique données 3+ mois
Scénario	1. Gestionnaire accède dashboard Analytics 2. Sélectionne période (3 derniers mois) 3. Consulte KPIs principaux : - Taux débordements : 1.8% (objectif <2% ✅) - Distance moyenne tournées : -18% vs N-1 - Coûts : -15 000€/mois économisés - Émissions CO_2 : -12 tonnes/mois 4. Analyse graphiques évolution temporelle 5. Identifie zone problématique (Zone Nord : 4% débordements) 6. Drill-down sur Zone Nord 7. Décide action : augmenter fréquence collecte 8. Génère rapport PDF pour direction
Résultat	Décision data-driven, rapport généré

IV. SPÉCIFICATIONS FONCTIONNELLES DÉTAILLÉES

Cette section décrit en détail les fonctionnalités de chaque module de la plateforme ECOTRACK.

4.1 Module Gestion des Conteneurs

Description :

Gestion complète du cycle de vie des conteneurs : création, modification, géolocalisation, historique remplissage, maintenance, alertes.

Fonctionnalités :

CRUD Conteneurs

- • Créer nouveau conteneur (capacité, type, position GPS)
- • Modifier attributs (capacité, type, zone)
- • Supprimer conteneur (soft delete)
- • Rechercher/filtrer (zone, type, statut, remplissage)

Géolocalisation

- • Saisie coordonnées GPS (latitude, longitude)
- • Sélection position sur carte interactive
- • Validation position (dans zone de collecte)
- • Affichage carte avec tous conteneurs
- • Clustering markers si >100 conteneurs

Monitoring Remplissage

- • Affichage taux remplissage temps réel
- • Code couleur : Vert (<50%), Orange (50-80%), Rouge (>80%)
- • Historique remplissage (graphique 30 jours)
- • Prédiction remplissage (ML - optionnel M1 Data)
- • Alertes automatiques si >80%

Maintenance

- • Marquer conteneur "En maintenance"
- • Historique interventions
- • Signalement problème (endommagé, sale)
- • Planning maintenance préventive

Import/Export

- • Import CSV conteneurs (création lot)
- • Export CSV liste conteneurs
- • Export GeoJSON pour SIG
- • Template CSV fourni

Règles métier :

- → Identifiant unique obligatoire (format : C-XXXX)
- → Capacité entre 100L et 5000L
- → Types autorisés : Ordures, Recyclage, Verre, Compost
- → Position GPS obligatoire et dans périmètre métropole
- → Un conteneur appartient à exactement une zone
- → Taux remplissage 0-100%
- → Historique conservé 12 mois
- → Alerte automatique si >80% + pas collecté 48h

4.2 Module Gestion des Zones

Fonctionnalités clés :

- ✓ Création zones géographiques avec polygones
- ✓ Assignation conteneurs à zones
- ✓ Statistiques par zone (nb conteneurs, taux moyen)
- ✓ Carte choroplèthe taux remplissage
- ✓ Import/export zones (GeoJSON, Shapefile)

4.3 Module Optimisation des Tournées

Fonctionnalités clés :

- ✓ Algorithme Nearest Neighbor (rapide)
- ✓ Algorithme 2-opt (optimal)
- ✓ Sélection conteneurs intelligente (remplissage >70%)
- ✓ Calcul distance totale et durée estimée
- ✓ Visualisation tournée sur carte
- ✓ Export feuille de route PDF
- ✓ Assignation tournée à agent
- ✓ Comparaison algorithmes (tableau perf)

4.4 Module IoT et Capteurs

Fonctionnalités clés :

- ✓ Simulateur capteurs MQTT (Python)
- ✓ Publication mesures toutes les 5min
- ✓ Validation données reçues
- ✓ Mise à jour BDD temps quasi-réel
- ✓ Gestion capteurs défaillants
- ✓ Dashboard monitoring IoT
- ✓ Logs flux MQTT

4.5 Module Monitoring Temps Réel

Fonctionnalités clés :

- ✓ Dashboard live (auto-refresh 30s)
- ✓ Carte temps réel tous conteneurs
- ✓ Liste alertes actives
- ✓ Notifications push débordements
- ✓ Statut système (nb capteurs actifs)

- ✓ Graphique évolution 24h
- ✓ Filtres temps réel (zone, type, statut)

4.6 Module Analytics et Reporting

Fonctionnalités clés :

- ✓ 8+ types graphiques (line, bar, pie, heatmap, etc.)
- ✓ KPIs principaux (nb collectes, distance totale, coûts)
- ✓ Évolution temporelle (jour, semaine, mois)
- ✓ Top 10 conteneurs critiques
- ✓ Taux remplissage moyen par zone
- ✓ Corrélations spatiotemporelles
- ✓ Génération rapports PDF automatiques
- ✓ Export Excel avec graphiques
- ✓ Scheduler rapports (hebdo, mensuel)

4.7 Module Gamification Citoyenne

Fonctionnalités clés :

- ✓ Système points (actions : signalement +10, tri +5)
- ✓ 30+ badges progressifs (Bronze, Argent, Or)
- ✓ Niveaux utilisateurs (1-10)
- ✓ Défis hebdomadaires (réduire déchets 10%)
- ✓ Leaderboard (top 50)
- ✓ Récompenses virtuelles
- ✓ Notifications achievements
- ✓ Profil gamification détaillé
- ✓ Statistiques impact environnemental personnel

4.8 Module Authentification et Droits

Fonctionnalités clés :

- ✓ Inscription/connexion (email + password)
- ✓ JWT tokens (access + refresh)
- ✓ MFA via TOTP (optionnel)
- ✓ RBAC : Citoyen, Agent, Gestionnaire, Admin
- ✓ SSO intégration (optionnel)
- ✓ Gestion sessions
- ✓ Password policies (12+ chars, complexité)
- ✓ Rate limiting login

- ✓ Logs authentification

4.9 Module Support et Communication

Fonctionnalités clés :

- ✓ Système tickets support
- ✓ Catégories : Technique, Collecte, Gamification
- ✓ Priorités : Basse, Normale, Haute, Urgente
- ✓ Chatbot FAQ basique
- ✓ Notifications (email, in-app)
- ✓ Historique tickets utilisateur
- ✓ Réponses agents support
- ✓ SLA tracking

4.10 Module Administration

Fonctionnalités clés :

- ✓ Dashboard admin (stats globales)
- ✓ Gestion utilisateurs (CRUD, RBAC, désactivation)
- ✓ Gestion conteneurs avancée (import lot, modif masse)
- ✓ Gestion zones géographiques
- ✓ Logs système (connexions, actions, erreurs)
- ✓ Modération signalements
- ✓ Configuration plateforme (paramètres)
- ✓ Export données (CSV, JSON)
- ✓ Audit trail (who, what, when)

V. ARCHITECTURE SYSTÈME

5.1 Architecture Globale

ECOTRACK suit une architecture moderne en couches, séparant frontend, backend, données et services externes.

```
┌──────────────────────────────────────────────────────────────┐
│                    UTILISATEURS                              │
│     (Citoyens, Agents, Gestionnaires, Admins)                │
└──────────────────────────────────────────────────────────────┘
                        │ HTTPS
                        ▼
          ┌──────────────────────────────┐
          │   FRONTEND (SPA)             │
          │   React/Vue + TypeScript     │
          │                              │
          │  - Routes & Navigation       │
          │  - State Management          │
          │  - Components UI             │
          │  - Services API              │
          └──────────────────────────────┘
                        │ REST API + WebSocket
                        ▼
          ┌──────────────────────────────┐
          │   BACKEND API                │
          │   Node.js/Python             │
          │                              │
          │  ┌────────────────────┐      │
          │  │ Controllers        │      │ ← Gestion requêtes
          │  └────────────────────┘      │
          │           │                  │
          │           ▼                  │
          │  ┌────────────────────┐      │
          │  │ Services (Métier)  │      │ ← Logique métier
          │  └────────────────────┘      │
          │           │                  │
          │           ▼                  │
          │  ┌────────────────────┐      │
          │  │ Repositories       │      │ ← Accès données
          │  └────────────────────┘      │
          └──────────────────────────────┘
                        │
          ┌─────────────┼────────────────┐
          │           ▼                  │
          │  ┌────────────────┐          │
          │  │ PostgreSQL     │          │
          │  │ + PostGIS      │          │
          │  └────────────────┘          │
          │                              │
          │  ┌────────────────┐          │
          │  │ Redis (Cache)  │          │
          │  └────────────────┘          │
          └──────────────────────────────┘

              │
        ┌─────┼──────────────────┐
        │                        │
        ▼                        ▼
```

```
┌──────────────┐     ┌──────────────┐
│ MQTT Broker  │     │ Services     │
│ (Mosquitto)  │     │ Externes     │
│              │     │              │
│ Capteurs IoT │     │ - Maps API   │
└──────────────┘     │ - Email      │
                     └──────────────┘
```

5.2 Architecture Applicative

Principe : Clean Architecture / Architecture Hexagonale

Le backend suit les principes de Clean Architecture pour séparer les responsabilités et faciliter tests et maintenabilité.

Couche Présentation

- • Controllers : Réception requêtes HTTP
- • Middlewares : Authentification, validation
- • WebSocket : Communication temps réel
- • DTOs : Data Transfer Objects

Couche Application

- • Services : Logique métier
- • Use Cases : Cas d'utilisation métier
- • Orchestration : Coordination services
- • Validation : Règles métier

Couche Domaine

- • Entities : Modèles métier (Container, Zone, Route)
- • Business Rules : Règles métier pures
- • Interfaces : Contrats (Repository interfaces)
- • Value Objects : Objets valeur immutables

Couche Infrastructure

- • Repositories : Implémentation accès BDD
- • Database : Configuration ORM
- • External APIs : Services externes
- • File System : Gestion fichiers

5.3 Architecture Données

Base de données : PostgreSQL 14+ avec PostGIS

PostgreSQL est choisi pour sa robustesse, son support des transactions ACID, et l'extension PostGIS pour les données géospatiales.

Tables principales :

users

- Utilisateurs (id, email, password_hash, role, created_at)

containers

- Conteneurs (id, name, capacity, type, position GEOMETRY, zone_id, fill_level)

zones

- Zones (id, name, polygon GEOMETRY, city_area)

routes

- Tournées (id, name, date, status, distance_km, duration_min, path GEOMETRY)

route_steps

- Étapes tournées (id, route_id, container_id, sequence, estimated_time)

measurements

- Mesures IoT (id, container_id, fill_level, temperature, timestamp)

badges

- Badges (id, name, description, icon, criteria)

user_badges

- Attribution badges (user_id, badge_id, earned_at)

tickets

- Tickets support (id, user_id, category, priority, status, description)

notifications

- Notifications (id, user_id, type, message, read, created_at)

5.4 Architecture Réseau et Sécurité

Composants réseau (spécialité Cyber) :

- • DMZ : Serveurs web publics (HTTPS)
- • Zone Privée : Backend API, BDD PostgreSQL
- • Zone Management : Monitoring (Prometheus, Grafana)
- • Firewall pfSense : Filtrage trafic
- • IDS/IPS Suricata : Détection intrusions
- • VPN IPsec : Accès admin sécurisé
- • Load Balancer : Répartition charge (HA)

Mesures sécurité :

- 🔒 HTTPS obligatoire (TLS 1.3)
- 🔑 Authentification JWT avec refresh tokens
- 👤 RBAC granulaire (4 rôles)
- 🛡️ Protection OWASP Top 10
- 🔐 Chiffrement passwords (bcrypt)
- 🚫 Rate limiting (100 req/min/IP)
- 📝 Logs centralisés (audit trail)
- 🔍 Scan vulnérabilités (Trivy)
- ⚡ WAF (Web Application Firewall)
- 📊 Monitoring sécurité (SIEM Wazuh pour Cyber)

VI. MODÈLE DE DONNÉES

Cette section présente les entités principales et leurs relations.

```
ENTITÉS PRINCIPALES ET RELATIONS :

┌──────────────┐          ┌──────────────┐          ┌──────────────┐
│   USERS      │          │ CONTAINERS   │          │   ZONES      │
├──────────────┤          ├──────────────┤          ├──────────────┤
│ id PK        │          │ id PK        │          │ id PK        │
│ email        │          │ name         │──────────│ name         │
│ password     │          │ capacity     │  N:1     │ polygon      │
│ role         │          │ type         │          │ city_area    │
│ points       │          │ position     │          └──────────────┘
└──────────────┘          │ fill_level   │
       │                  │ zone_id FK   │
       │                  └──────────────┘
       │                         │
       │ 1:N                     │ N:M
       │                         │
       ▼                         ▼
┌──────────────┐          ┌──────────────┐
│   BADGES     │          │   ROUTES     │
├──────────────┤          ├──────────────┤
│ id PK        │          │ id PK        │
│ name         │◄─────────│ name         │
│ criteria     │   N:M    │ date         │
└──────────────┘          │ distance     │
                          │ path         │
                          └──────────────┘
                                 │
                                 │ 1:N
                                 │
                                 ▼
                          ┌──────────────┐
                          │ ROUTE_STEPS  │
                          ├──────────────┤
                          │ route_id FK  │
                          │ container_id │
                          │ sequence     │
                          └──────────────┘

MEASUREMENTS ──N:1──> CONTAINERS
TICKETS ──N:1──> USERS
NOTIFICATIONS ──N:1──> USERS
```

VII. EXIGENCES NON FONCTIONNELLES

7.1 Performance

- ✓ Temps réponse API : < 500ms (95e percentile)
- ✓ Requêtes SQL : < 200ms pour 90% requêtes
- ✓ Chargement page : < 3s (Lighthouse)
- ✓ Concurrent users : Support 500 utilisateurs simultanés
- ✓ IoT throughput : 2000 mesures/5min = 400 msg/min
- ✓ Cache hit ratio : > 80% pour données fréquentes

7.2 Sécurité

- ✓ Conformité OWASP Top 10
- ✓ Chiffrement données sensibles (HTTPS, bcrypt)
- ✓ Authentification forte (JWT + MFA optionnel)
- ✓ RBAC avec moindre privilège
- ✓ Audit trail complet
- ✓ Scan vulnérabilités régulier
- ✓ Backup chiffré quotidien
- ✓ Conformité RGPD (basique)

7.3 Disponibilité

- ✓ Uptime : 99.5% visé (43.8h downtime/an max)
- ✓ RTO (Recovery Time Objective) : < 4h
- ✓ RPO (Recovery Point Objective) : < 1h
- ✓ Backup automatique quotidien
- ✓ Monitoring 24/7
- ✓ Plan PCA (Plan Continuité Activité)

7.4 Scalabilité

- ✓ Horizontal scaling : Ajout serveurs sans downtime
- ✓ Database partitioning : Tables measurements par mois
- ✓ Caching : Redis pour sessions et données fréquentes
- ✓ CDN : Actifs statiques (images, CSS, JS)
- ✓ Load balancing : Répartition charge
- ✓ Future 10 000 conteneurs supportés

7.5 Ergonomie et Accessibilité

- ✓ Responsive design : Desktop, tablet, mobile
- ✓ WCAG 2.1 niveau AA
- ✓ Temps chargement : < 3s
- ✓ Navigation intuitive
- ✓ Feedback utilisateur clair
- ✓ Aide contextuelle
- ✓ Internationalisation (i18n) : FR, EN prévu
- ✓ Dark mode (optionnel)

VIII. INTERFACES ET INTÉGRATIONS

8.1 Interfaces Utilisateurs

Interface Citoyen (Web + Mobile)

- • Page d'accueil : Carte conteneurs + stats personnelles
- • Tableau de bord : Points, badges, niveau, défis
- • Carte interactive : Visualiser conteneurs proches
- • Signalements : Formulaire + photo + géolocalisation
- • Profil : Historique actions, achievements
- • Notifications : Liste + préférences
- • FAQ et Support : Chatbot + tickets

Interface Agent Collecte (Mobile)

- • Tournée du jour : Liste ordonnée + carte
- • Navigation GPS : Guidage vers prochain conteneur
- • Scan QR code : Marquer collecté
- • Signalement terrain : Problème + photo
- • Historique : Tournées passées
- • Profil : Stats personnelles (km, collectes)

Interface Gestionnaire (Web)

- • Dashboard principal : KPIs + alertes
- • Gestion conteneurs : CRUD + carte + import CSV
- • Gestion zones : Édition polygones + stats
- • Optimisation tournées : Assistant création + algo
- • Planning : Calendrier tournées + assignation
- • Analytics : 8+ graphiques + exports
- • Rapports : Génération PDF/Excel
- • Utilisateurs : Consultation + stats engagement

Interface Admin (Web)

- • Dashboard admin : Stats globales système
- • Gestion utilisateurs : CRUD + RBAC + désactivation
- • Gestion conteneurs : Import masse + modifications
- • Configuration : Paramètres plateforme
- • Logs système : Consultation + filtres

- • Modération : Signalements + tickets
- • Monitoring : Status services + métriques
- • Audit : Historique actions critiques

8.2 API REST

L'API REST est le cœur de la communication entre frontend et backend. Elle suit les principes RESTful et est documentée via OpenAPI/Swagger.

Endpoints principaux :

Authentification

- POST /api/auth/register - Inscription
- POST /api/auth/login - Connexion
- POST /api/auth/refresh - Refresh token
- POST /api/auth/logout - Déconnexion
- GET /api/auth/me - Profil utilisateur

Conteneurs

- GET /api/containers - Liste conteneurs (filtres, pagination)
- GET /api/containers/:id - Détail conteneur
- POST /api/containers - Créer conteneur
- PUT /api/containers/:id - Modifier conteneur
- DELETE /api/containers/:id - Supprimer conteneur
- GET /api/containers/:id/history - Historique remplissage
- POST /api/containers/import - Import CSV

Zones

- GET /api/zones - Liste zones
- GET /api/zones/:id - Détail zone
- POST /api/zones - Créer zone
- PUT /api/zones/:id - Modifier zone
- GET /api/zones/:id/stats - Statistiques zone

Tournées

- GET /api/routes - Liste tournées
- GET /api/routes/:id - Détail tournée
- POST /api/routes/optimize - Optimiser tournée
- POST /api/routes - Créer tournée
- PUT /api/routes/:id - Modifier tournée
- GET /api/routes/:id/export - Export PDF

Analytics

- GET /api/analytics/kpis - KPIs principaux
- GET /api/analytics/containers/top - Top conteneurs critiques

- GET /api/analytics/zones/heatmap - Données heatmap
- GET /api/analytics/evolution - Évolution temporelle
- POST /api/analytics/report - Générer rapport

Gamification

- GET /api/gamification/profile - Profil gamification
- GET /api/gamification/badges - Liste badges
- GET /api/gamification/leaderboard - Classement
- POST /api/gamification/challenges/:id/join - Rejoindre défi

Support

- GET /api/tickets - Liste tickets
- POST /api/tickets - Créer ticket
- GET /api/tickets/:id - Détail ticket
- PUT /api/tickets/:id - Mettre à jour ticket

Conventions API :

- → Format : JSON pour requêtes et réponses
- → Status codes HTTP standard (200, 201, 400, 401, 404, 500)
- → Pagination : query params ?page=1&limit=20
- → Filtres : query params ?zone=1&type=Recyclage
- → Tri : query param ?sort=fillLevel:desc
- → Authentification : Bearer token JWT dans header Authorization
- → Rate limiting : 100 req/min par IP
- → Versioning : /api/v1/ (prévu pour évolutions)
- → Documentation : Swagger UI à /api/docs

8.3 Intégrations Externes

OpenStreetMap / Leaflet

- Cartographie interactive, tuiles de carte, geocoding

MQTT Broker (Mosquitto)

- Communication IoT capteurs → plateforme

Service Email (SMTP)

- Envoi notifications, rapports, alertes

Service SMS (optionnel)

- Notifications critiques agents

API Météo (optionnel)

- Corrélation remplissage / météo

Export PDF (jsPDF/PDFKit)

- Génération feuilles route, rapports

Export Excel (ExcelJS)

- Export données analytiques

IX. ORGANISATION ET MÉTHODOLOGIE

9.1 Méthodologie Agile - Scrum

Le projet ECOTRACK suit la méthodologie Agile Scrum avec des sprints de 2 semaines. Cela permet une livraison incrémentale et une adaptation continue aux besoins.

Organisation Scrum :

Product Backlog : Liste priorisée de toutes les user stories

Sprint Planning : Réunion début sprint : sélection stories, estimation

Daily Standup : 15 min quotidien : hier, aujourd'hui, blocages

Sprint Review : Démo fin sprint aux parties prenantes

Sprint Retrospective : Amélioration continue : what went well, what to improve

Sprint Duration : 2 semaines (8 sprints sur 4 mois)

Definition of Done : Code testé, documenté, reviewé, déployé staging

9.2 Organisation Équipes

Les étudiants Master 1 travaillent en équipes de 2-3 personnes maximum, avec spécialisation selon leur filière mais collaboration inter-spécialités encouragée.

Répartition par spécialité :

Spécialité	Focus principal	Collaboration avec
Cyber-Réseaux	Infrastructure, sécurité, monitoring, réseau multi-zones	Dev (API), Data (BDD)
Développement	Backend API, Frontend, Algorithmes, CI/CD, Tests	Cyber (déploiement), Data (analytics)
Data Science	PostGIS, Analytics, ML prédictif, DataViz, ETL	Dev (API data), Cyber (monitoring)

Outils collaboratifs :

- • Git + GitHub : Versioning code, pull requests, issues
- • GitHub Projects : Backlog, Kanban board, sprints
- • Discord/Slack : Communication équipe

- • Google Docs : Documentation collaborative
- • Figma : Maquettes UI/UX
- • Draw.io : Diagrammes architecture
- • Postman : Tests API, documentation

9.3 Planning et Jalons (4 mois)

Période	Sprints	Objectifs	Livrable
Janvier S1-2	1-2	Setup, Architecture, BDD	Livrable 1 : Conception (25 pts)
Janvier S3-4	3-4	CRUD Conteneurs/Zones, Auth basique	Livrable 2 : MVP Backend (20 pts)
Février S1-2	5-6	Frontend pages principales, Carte	Sprint Review
Février S3-4	7-8	IoT simulateur, Optimisation tournées	Livrable 3 : Features core (30 pts)
Mars S1-2	9-10	Analytics, Gamification, Tests	Sprint Review
Mars S3-4	11-12	CI/CD, Monitoring, Sécurité avancée	Livrable 4 : Sécurité+Analytics (25 pts)
Avril S1-2	13-14	Backoffice, Documentation, Tests finaux	Sprint Review
Avril S3-4	15-16	Finalisation, Déploiement, Soutenance	Livrable Final + Soutenance (100 pts)

Jalons critiques :

- 📅 S2 : Livrable 1 Conception

  25 points - Architecture, MCD, Wireframes

- 📅 S4 : Livrable 2 MVP

  20 points - CRUD conteneurs, Auth, BDD

- 📅 S8 : Livrable 3 Features Core

  30 points - Tournées optimisées, IoT, Frontend

- 📅 S12 : Livrable 4 Sécurité+Analytics

  25 points - Dashboards, Tests, CI/CD

- 📅 S16 : Livrable Final

  100 points - Projet complet + DCT + Soutenance

X. LIVRABLES ET DOCUMENTATION

10.1 Livrables Techniques

Livrables obligatoires :

Livrable	**Contenu**
Code source	Repository Git complet (frontend + backend)
Base de données	Scripts SQL DDL, seeds, migrations
Application déployée	URL staging accessible + URL production
Tests automatisés	>60% couverture (Dev), Suite tests validée
Pipeline CI/CD	GitHub Actions fonctionnel, badges
Documentation technique	DCT 30-35 pages spécialité
Documentation API	Swagger/OpenAPI complet
Guide utilisateur	Manuel utilisateur par profil
Guide installation	README détaillé, Docker Compose
Présentation	Slides soutenance + démo vidéo 5min

10.2 Documentation Requise

DCT (Document Conception Technique)

- • Présentation projet et contexte
- • Architecture globale (diagrammes)
- • Modèle de données (MCD, MLD)
- • Spécifications techniques détaillées
- • Algorithmes implémentés (pseudo-code, complexité)
- • Choix technologiques justifiés
- • Tests et qualité
- • Déploiement et exploitation
- • Perspectives d'évolution
- • Format : 30-35 pages PDF

README.md

- • Description projet
- • Prérequis (Node, PostgreSQL, Docker)
- • Installation étape par étape
- • Configuration (.env template)
- • Lancement (dev, prod)
- • Tests (npm test, coverage)
- • Déploiement
- • Architecture overview
- • Contribution guidelines
- • License et contacts

Documentation API

- • OpenAPI 3.0 specification
- • Swagger UI accessible
- • Description endpoints
- • Exemples requêtes/réponses
- • Codes erreurs
- • Authentification
- • Rate limiting

Guide Utilisateur

- • Guides par profil (Citoyen, Agent, Gestionnaire)
- • Captures d'écran annotées
- • Tutoriels pas à pas
- • FAQ
- • Troubleshooting
- • Format : PDF ou Wiki

10.3 Critères d'Acceptation

Le projet est validé si et seulement si :

✅ **Note technique**

- ≥ 78/130 points (60%)

✅ **Application fonctionnelle**

- Déployée et accessible, pas de bugs bloquants

✅ **Modules obligatoires**

- Tous les modules MUST sont implémentés

✅ **Tests**

- >60% couverture (Dev), Suite tests passe (Cyber/Data)

✅ **Documentation**

- DCT complet + README + API docs

✅ **Livrables intermédiaires**

- Les 4 livrables rendus à temps

✅ **Soutenance**

- ≥ 18/30 points, démo réussie

✅ **Code quality**

- Lint passe, pas de code smell critique (SonarQube)

✅ **Git**

- Historique commits propre, branches, pull requests

✅ **Déploiement**

- Application en production, accessible publiquement

⚠️ **Causes de non-validation :**

- ❌ Note < 60%
- ❌ Application non déployée ou inaccessible
- ❌ Bugs critiques non corrigés
- ❌ Documentation incomplète (< 25 pages DCT)

- ❌ Plagiat détecté
- ❌ Absence à la soutenance
- ❌ Livrables intermédiaires non rendus (>2)
- ❌ Tests < 40% couverture (Dev)
- ❌ Code non versionné ou historique Git inexistant

XI. ANNEXES

11.1 Glossaire

API : Application Programming Interface - Interface de programmation

BDD : Base De Données

CDC : Cahier Des Charges

CI/CD : Continuous Integration / Continuous Deployment

CRUD : Create, Read, Update, Delete

DCT : Document de Conception Technique

DMZ : DeMilitarized Zone - Zone démilitarisée

DTO : Data Transfer Object

ETL : Extract, Transform, Load

GPS : Global Positioning System

IDS/IPS : Intrusion Detection/Prevention System

IoT : Internet of Things - Internet des Objets

JWT : JSON Web Token

KPI : Key Performance Indicator

MFA : Multi-Factor Authentication

ML : Machine Learning

MQTT : Message Queuing Telemetry Transport

ORM : Object-Relational Mapping

PCA : Plan de Continuité d'Activité

PostGIS : Extension spatiale pour PostgreSQL

PWA : Progressive Web App

RBAC : Role-Based Access Control

REST : Representational State Transfer

RGPD : Règlement Général sur la Protection des Données

RNCP : Répertoire National des Certifications Professionnelles

ROI : Return On Investment

RTO/RPO : Recovery Time/Point Objective

SIEM : Security Information and Event Management

SLA : Service Level Agreement

SPA : Single Page Application

SQL : Structured Query Language

SRID : Spatial Reference System Identifier

TSP : Traveling Salesman Problem

UX/UI : User Experience / User Interface

VPN : Virtual Private Network

WCAG : Web Content Accessibility Guidelines

11.2 Références

Documentation technique :

- React : https://react.dev
- Vue.js : https://vuejs.org
- Node.js : https://nodejs.org
- PostgreSQL : https://www.postgresql.org
- PostGIS : https://postgis.net
- Leaflet : https://leafletjs.com
- Docker : https://docs.docker.com
- GitHub Actions : https://docs.github.com/en/actions
- OpenAPI : https://swagger.io/specification
- MQTT : https://mqtt.org

Normes et standards :

- OWASP Top 10 : https://owasp.org/www-project-top-ten
- WCAG 2.1 : https://www.w3.org/WAI/WCAG21/quickref
- RGPD : https://www.cnil.fr
- ISO 27001 : https://www.iso.org/isoiec-27001-information-security.html
- REST API Best Practices : https://restfulapi.net

Algorithmes et optimisation :

- TSP (Traveling Salesman Problem) : Wikipedia TSP
- 2-opt algorithm : https://en.wikipedia.org/wiki/2-opt
- Nearest Neighbor : Algorithmes gloutons
- Haversine formula : Calcul distance GPS

11.3 Contacts

Rôle	Nom	Contact
Responsable pédagogique	École INGETIS	contact@ingetis.fr
Formateur Cyber-Réseaux	À définir	cyber@ingetis.fr
Formateur Développement	À définir	dev@ingetis.fr
Formateur Data Science	À définir	data@ingetis.fr

Support technique :

- 💬 Discord serveur : ECOTRACK M1 2025-2026
- 📧 Email support : support.ecotrack@ingetis.fr
- 📚 Documentation en ligne : https://docs.ecotrack.ingetis.fr
- 🐛 Issues GitHub : Repository du projet

CONCLUSION

ECOTRACK représente un projet d'envergure professionnelle qui permettra aux étudiants Master 1 de développer des compétences techniques avancées tout en contribuant à un projet à fort impact environnemental et sociétal.

Ce cahier des charges fonctionnel constitue la référence commune pour toutes les spécialités. Chaque spécialité dispose également de son cahier des charges exhaustif détaillant les 130-145 tâches spécifiques à réaliser.

🎯 Nous souhaitons à tous les étudiants un excellent projet et une réussite complète !

ECOTRACK - Cahier des Charges Fonctionnel Commun

Master : Cyber-Réseaux | Développement | Data Science

École INGETIS - Année 2025-2026

Version 3.0 - Janvier 2026

Document de référence pour toutes spécialités